

SUMMARY INFORMATION ABOUT TRANSFER LEARNING METHODS

Transfer Learning models are models that aim to shorten the training time and improve performance by applying the knowledge (weights) of previously trained models to similar or different problems.

Transfer Learning allows the information learned by pre-trained models—usually trained on large and comprehensive datasets—to be effectively used on new and relatively smaller datasets.

Models such as **VGG** and **ResNet** have been trained on millions of images and are therefore very good at identifying general image features.

MODEL BUILDING

For Example; let's assume we have built our manual CNN (Deep Learning) model as shown below. (Here, the systematics of model construction and the added layers are not important; this code is used for illustrative purposes only.)

```
inputs = keras.Input(shape = (224, 224, 3))
```

```
x = data_augmentation(inputs)
```

```
x = layers.Rescaling(1/255)(x)
```

```
x = layers.Conv2D(filters = 64, kernel_size = (3, 3), padding = "same", activation = "relu")(x)
```

```
x = layers.Conv2D(filters = 64, kernel_size = (5, 5), padding = "same", activation = "relu")(x)
```

```
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
```

```
x = layers.BatchNormalization()(x)
```

```
x = layers.Conv2D(filters=128, kernel_size=(3, 3), padding = "same", activation = "relu")(x)
```

```
x = layers.Conv2D(filters=128, kernel_size=(5, 5), padding = "same", activation = "relu")(x)
```

```
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
```

```
x = layers.BatchNormalization()(x)
```

```
x = layers.Conv2D(filters=128, kernel_size=(3, 3), padding = "same", activation = "relu")(x)
```

```
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
```

```
x = layers.BatchNormalization()(x)
```

```
x = layers.Conv2D(filters=64, kernel_size=(3, 3), padding = "same", activation = "relu")(x)
```

```
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
```

```
x = layers.BatchNormalization()(x)
```

```
x = layers.Conv2D(filters=64, kernel_size=(3, 3), padding = "same", activation = "relu")(x)
```

```
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
```

```
x = layers.BatchNormalization()(x)
```

```
x = layers.Dropout(0.4)(x)
```

```
#----ANN-----
```

```
x = layers.GlobalAveragePooling2D()(x) # Flatten yerine bunu kullandik
```

```
x = layers.BatchNormalization()(x)
```

```
outputs = layers.Dense(5, activation = "softmax")(x)
```

```
#---- Let's Build a Simple Model----
```

```
basic_model = keras.Model(inputs, outputs)
```

```
basic_model.summary()
```

Let's add the code that will run the basic model.

1. **Step:** Let this be the first execution phase. First, let's run it using a large value such as Learning Rate = 0.1 and a small value such as epoch = 5, so that the model learns quickly.

```
basic_model.compile(optimizer = optimizers.Adam(learning_rate = 0.1),  
                    loss = "sparse_categorical_crossentropy",  
                    metrics = ["accuracy"])
```

```
history = basic_model.fit(X_train, y_train, epochs = 5,  
                          validation_data = (X_test, y_test),  
                          callbacks = [early_stop, best_model])
```

2. **Step:** Let's set Learning Rate = 0.01 and epoch = 5 in this step.

```
basic_model.compile(optimizer = optimizers.Adam(learning_rate = 0.01),  
                    loss = "sparse_categorical_crossentropy",  
                    metrics = ["accuracy"])
```

```
history = basic_model.fit(X_train, y_train, epochs = 5,  
                          validation_data = (X_test, y_test),  
                          callbacks = [early_stop, best_model])
```

The logic of pre-trained models comes into play precisely at this point. The model will retain the weights it learned in the first phase and will start the second phase using these weights.

3. Step: At this stage, let's set the Learning Rate to 0.001 and the epoch to 50. This is exactly what happens when we use pre-trained models—known as Transfer Learning models—in our own work: the model transfers the weights (or we can say knowledge) it learned previously (trained on millions of data points, with very low learning rates and very high epochs) into our model.

This is where the concept of Transfer Learning comes from: the transfer, sharing, or carrying over of previously learned information into our own models.

During the transfer of weights, these weights can either be **kept fixed or fine-tuned along with the new data**. Fine-tuning allows **the weights to be slightly adjusted** so that the model can better handle the new task. However, this process requires **careful attention** and **adherence to certain best practices**. Otherwise, **incorrect applications can negatively affect model performance** and lead to difficulties in obtaining the expected results.

Applying Transfer Learning:

1. Freezing the Model's Weights

In Deep Learning analyses, when using Transfer Learning models, it is considered a **best practice** to **initially freeze the weights** of the Transfer Learning model (i.e., prevent them from being updated or retrained).

This can be seen in the code examples below.

```
model = applications.Xception(input_shape = (224, 224, 3), include_top=False)
# In the first line of code, the include_top parameter should be set as False, otherwise it won't use Transfer Learning.
model.trainable = False
```

In the second line of this code block, by writing **model.trainable = False**, we **froze all the layers** of the Transfer Learning model (here, Xception).

In this way, we ensured that our Deep Learning model **uses the previously learned information (weights) without changing them**.

2. Adding extra CONV layer

Additional layers can be added, but the possibility of overfitting should be taken into consideration.

You can see an example of this in the code block below:

```
# I manually added these layers to the Deep Learning model myself to improve the scores
```

```
x = layers.Conv2D(filters=64, kernel_size=(3, 3), padding = "same", activation = "relu")(x)
```

```
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
```

```
x = layers.BatchNormalization()(x) x = layers.Dropout(0.4)(x)
```

```
# But I got train accuracy = 1, validation accuracy = 0.89
```

```
# In other words, overfitting occurred. Therefore, it is not necessary for us to add additional layers to Transfer Learning models ourselves.
```

If we do, it's important to pay attention to the issue of overfitting.

3. Opening the last some layers for some changes.

If desired, we can unfreeze (make trainable) a certain number of layers from the end of the Transfer Learning model for retraining.

The reason for unfreezing the final layers is that the model learns progressively through its layers, and the final layers contain a certain level of accumulated knowledge.

```
#To unfreeze the last 2 layers of the Xception model:
```

```
for i in base_model.layers[-2:]:
    i.trainable = True
```

In this way, we can unfreeze as many of the final layers as we want (in this code block, the last 2 layers) from the existing Transfer Learning model (here, Xception) for training in our newly built Deep Learning model.

We specifically emphasize "from the end" because, as we learned earlier, models progress by learning layer by layer.

By modifying the last layers, we are trying to further improve the existing training. This process—unfreezing the final layers—is not guaranteed to improve the scores, but you can try it if you wish.

4. Fine tuning

If we have a pre-trained model, we can customize it for our own tasks.

The fine-tuning process allows us to train the final layers of the pre-trained model on our own dataset.

This way, the model retains its ability to capture general features while also learning the specific features needed for your task.

The fine-tuning process can be carried out as shown in the code block below.

```
base_model.trainable = True early_stop = keras.callbacks.EarlyStopping(monitor = "val_loss", patience= 10)

model.compile(optimizer = optimizers.Adam(learning_rate = 0.00001),
loss = "sparse_categorical_crossentropy", metrics = ["accuracy"])

model.fit(X_train, y_train, epochs = 100,
        validation_data = (X_test, y_test),
        callbacks = [early_stop, best_model])
```

In this code block, by setting `base_model.trainable = True`, we allowed the base model to use the weights at the point where `val_loss` was minimized, and in a sense, we opened the final layer of the model for training.

The important point to note here is that the Deep Learning model should first be run with the existing weights of the Transfer Learning model, and the fine-tuning process should be carried out afterward.

This allows us to explore ways to further train/improve the model that was already trained with those weights.

With this process, we are also training the final layer of the existing Transfer Learning model on our own dataset.

Let us emphasize once again that this process is not guaranteed to improve the scores, but it is an option worth trying.

USEFUL ROADMAP Suggestions:

1) If the image quality is good, apply as many options as possible in Image Generation when building a manual CNN so that the model learns better (because a manual CNN does not have pre-trained weights). **If the image quality is poor, it might be better not to do this—try and see.**

2) When using Transfer Learning (TL) models, you can reduce the options in Image Generation, since pre-trained weights already exist—no need to slow down the model unnecessarily.

3) I applied Image Generation only to the training data, not to the test data.

This is required by the architecture used in this notebook.

However, if desired, it is also possible to apply Image Generation to test data.

4) "Do not perform rescaling outside the model!" because some Transfer Learning models do this by default.

(Detailed explanation is provided below.)

5) Check the features of the TL (Transfer Learning) model you are going to use at <https://keras.io/api/applications/>.

If you see "include_preprocessing=True", it means rescaling is done internally (by default), and you don't need to do anything. (Explanation is provided below.)

6) First, run the model with a large Learning Rate (e.g., LR = 0.1) and a small number of Epochs (5–10). Check the scores. Then (as a second step), simply reduce the Learning Rate (e.g., to 0.01) in the model.fit() step, and train again with Epochs = 5–10. The model retains the weights from these intermediate steps in memory.

8) Then Fine Tuning;

a) LR=0.00001,

b) Trainable =True

c) Epoch = 100 getting the final result.

This method saves time and may also improve the scores.

POINTS TO CONSIDER WHEN APPLYING TRANSFER LEARNING

However, there are some important points to consider when using Transfer Learning models.

The success of the models that researchers build depends on choosing a Transfer Learning model that is compatible with their target analysis and making sure that their data is aligned with the type of data used during the training of the chosen Transfer Learning model.

The complete list of Transfer Learning models that can be used in Deep Learning is available on Keras' official site: <https://keras.io/api/applications/>.

New models are generally developed to address the shortcomings of earlier ones and to improve their speed and/or accuracy. In this regard, using newer models in analyses may increase success.

The first important point to consider is to examine how the data was fed into the Transfer Learning model during its original training and to convert your own data to that format accordingly.

Some Transfer Learning models handle image conversion within their own system (i.e., the preprocessing process is already built-in), while most others require the user to write additional code or provide the images in a format suitable for the model's system.

The second important point is: do not apply rescaling outside the model!

That means image color values should not be divided by 255 before being fed into the Transfer Learning model.

At this point, what needs to be done is to check the specifications of the Transfer Learning model to be used at <https://keras.io/api/applications/>.

If "include_preprocessing=True" is specified in the default settings, users do not need to perform rescaling themselves—this process is already handled by default.

Transfer Learning models are divided into two groups based on whether they perform the pre-processing operations automatically or not:

1. Models that performs Pre-Processing by Default :

EfficientNetV2B0 — **EfficientNetV2BL** and **ConvNeXtTiny-ConvNeXtXLarge** groups are examples of this. We can understand this by the documentation provided along with these models like:

EfficientNetV2B0 function

```
keras.applications.EfficientNetV2B0(  
    include_top=True,  
    weights="imagenet",  
    input_tensor=None,  
    input_shape=None,  
    pooling=None,  
    classes=1000,  
    classifier_activation="softmax",  
    include_preprocessing=True,  
    name="efficientnetv2-b0",  
)
```

As seen here, the parameter list includes the information "**include_preprocessing=True**".

Therefore, **rescaling is not necessary—raw data (i.e., the original form of the data) should be used in the analysis.**

2. Models where we have to perform Pre-Processing'i Manually:

In Manual CNN, Xception, VGG, ResNet, Inception, MobileNet, DenseNet, NasNet, and EfficientNetB0–B7, rescaling is not done by default (i.e., it is not embedded within the model).

When using these models, we ensure that the pre-processing is handled by the model itself by writing a single line of code. For example, when we look at the Xception model's documentation, this information is provided in the highlighted sections of the image.

Xception function

[source]

```
tf.keras.applications.Xception(  
    include_top=True,  
    weights="imagenet",  
    input_tensor=None,  
    input_shape=None,  
    pooling=None,  
    classes=1000,  
    classifier_activation="softmax",  
)
```

1

Instantiates the Xception architecture.

Reference

- Xception: Deep Learning with Depthwise Separable Convolutions (CVPR 2017)

For image classification use cases, see [this page](#) for detailed examples.

For transfer learning use cases, make sure to read the [guide to transfer learning & fine-tuning](#).

The default input image size for this model is 299x299.

2

3

Note: each Keras Application expects a specific kind of input preprocessing. For Xception, call `tf.keras.applications.xception.preprocess_input` on your inputs before passing them to the model. `xception.preprocess_input` will scale input pixels between -1 and 1.

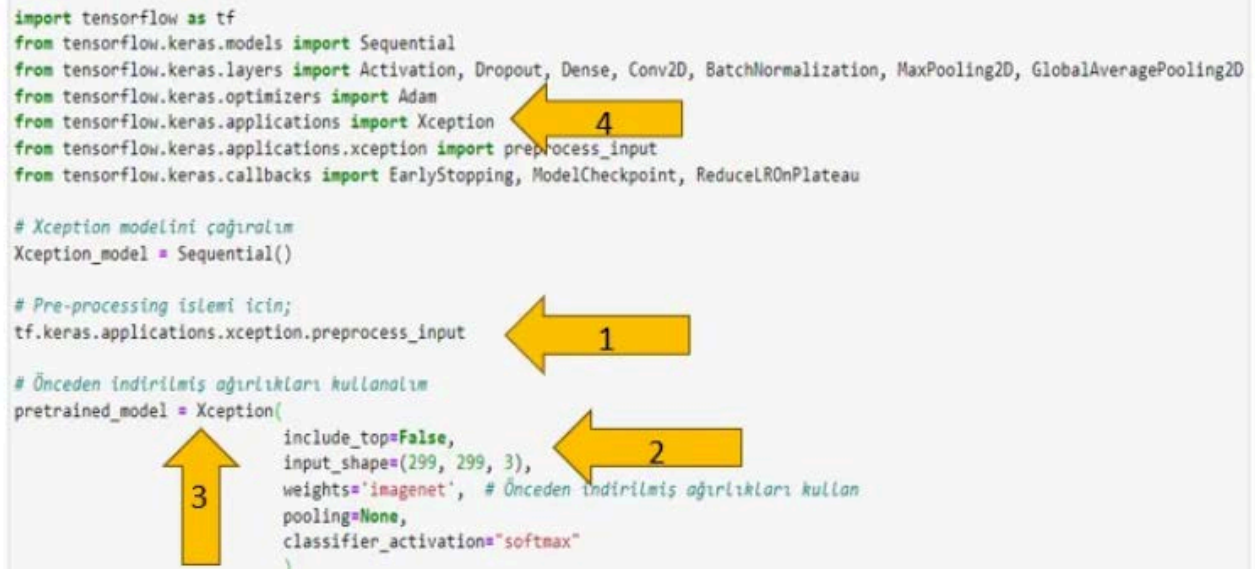
The default parameters do not include **"include_preprocessing=True"**.

However, in section 3 of the image, it shows *keras.applications.xception.preprocess_input*.

This means that when this line of code is added to the Deep Learning model being built, the pre-processing will be carried out in the way required by the Xception model.

Additionally, it is stated in section 2 that the images fed into the model must have dimensions of 299×299. This section is also pointed out by arrow number 2 in the image below.

An example related to adding this line of code is provided in the image below:



```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Activation, Dropout, Dense, Conv2D, BatchNormalization, MaxPooling2D, GlobalAveragePooling2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import Xception
from tensorflow.keras.applications.xception import preprocess_input
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

# Xception modelini çağıralım
Xception_model = Sequential()

# Pre-processing işlemi için;
tf.keras.applications.xception.preprocess_input

# Önceden indirilmiş ağırlıkları kullanalım
pretrained_model = Xception(
    include_top=False,
    input_shape=(299, 299, 3),
    weights='imagenet', # Önceden indirilmiş ağırlıkları kullan
    pooling=None,
    classifier_activation="softmax"
)
```

In the image, the line indicated by arrow number 1 shows the code that needs to be added when using the Xception Transfer Learning model.

When using other Transfer Learning models, the method name in the line indicated by arrow 1 must be changed accordingly.

To do this, it is helpful to visit the relevant Transfer Learning model's page at <https://keras.io/api/applications/> and get the necessary code from there.

A Complete Transfer Learning Model Workflow in Detail

Model: VGG

1. IMPORT THE NECESSARY LIBRARY

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Dropout
from tensorflow.keras.applications import VGG16
```

Explanation:

For transfer learning, we load the Keras and TensorFlow libraries, use ImageDataGenerator for data augmentation, and load the basic layers along with the pre-trained VGG16 model.

2. DATA AUGMENTATION AND PREPROCESSING

```
train_datagen = ImageDataGenerator(  
    rescale=1./255,      # Normalizing  
    rotation_range=20,   # Random Rotation  
    width_shift_range=0.2, # Random shift of width  
    height_shift_range=0.2, # Random shift of height  
    shear_range=0.15,    # Random Shear  
    zoom_range=0.2,      # Random zoom  
    horizontal_flip=True, # Random horizontal flip  
    fill_mode='nearest'   # pixel filling mode  
)
```

```
validation_datagen = ImageDataGenerator(rescale=1./255) # Apply only normalization on the  
validation or test data
```

Explanation:

We apply various data augmentation techniques to the training data, while only normalization is applied to the validation data.

3. CREATING FLOW OF DATA

```
train_dir = "data/train"  
validation_dir = "data/validation"
```

```
train_generator = train_datagen.flow_from_directory(  
    train_dir,  
    target_size=(224, 224), # VGG16 input dimension  
    batch_size=32,  
    class_mode='binary'     # Binary classification  
)
```

```
validation_generator = validation_datagen.flow_from_directory(  
    validation_dir,  
    target_size=(224, 224),  
    batch_size=32,  
    class_mode='binary'  
)
```

Explanation:

We create appropriate data flows for the model by resizing and labeling the images from the directory structure using the data augmentation objects.

Instead of validation_generator, we can also use test_generator.

4. LOAD THE PRE-TRAINED VGG16 MODEL

```
base_model = VGG16(  
    include_top=False, # We'll add our own (FC) top layers  
    weights='imagenet', # ImageNet trained weights  
    input_shape=(224, 224, 3) # input shape  
)
```

```
base_model.trainable = False # Freeze all layers so they are not retrained (feature extractor).
```

Explanation:

We remove the final (fully connected) layers. Feature extraction will be done using only the CONV layers. All layers are frozen by setting trainable=False.

5. BUILDING THE TRANSFER LEARNING MODEL

```
model = Sequential([
    base_model,          # VGG16 feature extractor
    Flatten(),           # Convert feature maps into a vector
    Dense(256, activation='relu'), # New FC Layer
    Dropout(0.5),        # Use dropout to prevent overfitting
    Dense(1, activation='sigmoid') # output layer, binary classification
])
```

Explanation:

We flatten the outputs of VGG16 and then add our own dense layers.

At the output, we use the sigmoid activation to obtain a probability between 0 and 1.

6. COMPILING THE MODEL

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Explanation:

Compiling the model with loss and metrics

7. MODEL TRAINING

```
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    epochs=10,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size
)
```

Explanation:

We train the model over the data flow for a specified number of epochs (e.g., 10).

The number of training and validation steps is automatically adjusted based on the amount of data.

8. MODEL EVALUATION AND PREDICTION

```
# Model's performance on the validation set
val_loss, val_acc = model.evaluate(validation_generator)
print(f"Validation Accuracy: {val_acc}")

# To make a prediction on a new image
import numpy as np
from tensorflow.keras.preprocessing import image
img = image.load_img("test.jpg", target_size=(224, 224))
img_array = image.img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)
prediction = model.predict(img_array)
print("Prediction result (probability):", prediction[0][0])
```

Explanation:

After training, the model's validation performance is measured.

To make a prediction on a new image, the image is preprocessed appropriately and then passed to the model.

SUMMARY

This workflow allows you to easily use the VGG16 model with transfer learning.

- Feature extraction is performed using the pre-trained model,
- You add your own final layers,
- The model is frozen, and only the newly added layers are trained,
- New data can easily be used for prediction.

Since each step includes both code and explanation, it helps you to practically learn and apply the logic of transfer learning!

Model: ResNET50

Using ResNet models such as ResNet50 for transfer learning can be done in a similar way to VGG16. Below is a step-by-step and well-explained code workflow that shows how to use the ResNet50 model for transfer learning.

1. IMPORTING THE NECESSARY LIBRARIES

```
import tensorflow as tf from tensorflow.keras.preprocessing.image
import ImageDataGenerator from tensorflow.keras.models
import Sequential from tensorflow.keras.layers
import Flatten, Dense, Dropout from tensorflow.keras.applications
import ResNet50
```

Explanation:

- Keras and TensorFlow libraries are necessary for building the model and processing the data.
 - We will use ResNet50 as a pre-trained model.
-

2. DATA AUGMENTATION AND PREPRICESSING

```
train_datagen = ImageDataGenerator(
    rescale=1./255,          # Normalize images to a range between 0 and 1
    rotation_range=20,       # Random rotation
    width_shift_range=0.2,   # Horizontal shift
    height_shift_range=0.2,  # Vertical shift
    shear_range=0.15,       # Random shear
    zoom_range=0.2,         # Random zoom
    horizontal_flip=True,    # Horizontal flip
    fill_mode='nearest'     # Pixel fill mode
)

validation_datagen = ImageDataGenerator(rescale=1./255) # Only Normalization
for the validation data
```

Explanation:

- Data augmentation techniques help the model learn with more data.
- Only normalization is applied to the validation set, because data augmentation is not performed on validation data.

3. DATA FLOW

```
train_dir = "data/train"
validation_dir = "data/validation"

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224),      # ResNet50 INPUT DIM
    batch_size=32,
    class_mode='binary'         # BINARY CLASSIFICATION
)

validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    target_size=(224, 224),      # ResNet50 INPUT DIMENTION
    batch_size=32,
    class_mode='binary'         # BINARY CLASSIFICATION
)
```

Explanation:

- The `flow_from_directory()` function loads the images from the specified folders and provides them according to the given size.
- `target_size=(224, 224)` specifies the input size expected by the ResNet50 model (images of size 224×224).

4. LOADING THE PRETRAINED RESNET50 MODEL

```
base_model = ResNet50(include_top=False,
                      # We do not include the top layers of ResNet50 (i.e., the
                      # fully connected layers).
                      weights='imagenet',      # Load the model trained with ImageNet weights.
                      input_shape=(224, 224, 3) # INPUT SHAPE
                      base_model.trainable = False # Freeze the weights of Pre-trained model
                      )
```

Explanation:

- `include_top=False`: The model's final layers (fully connected layers) are not included because we will add new classifier layers.
- `weights='imagenet'`: Uses the weights of the model trained on ImageNet.
- `base_model.trainable = False`: This line ensures that the pre-trained model's weights remain frozen.

We will only train the newly added layers.

5. ADDING OUR OWN MODEL LAYERS

```
model = Sequential([
    base_model,                # Adding the pre-trained ResNet50 model
    Flatten(),                # Converting feature maps into a vector

    Dense(1024, activation='relu'), # New dense layer, trainable
    Dropout(0.5),              # dropout layer to prevent overfitting
    Dense(1, activation='sigmoid') # output layer for (binary classification
    sigmoid)
])
```

Explanation:

- **Flatten():** Converts the 3D output from convolutional layers into a single 1D vector.
 - **Dense(1024, activation='relu'):** Adds a new fully connected layer.
 - **Dropout(0.5):** Uses 50% dropout to prevent overfitting.
 - **Dense(1, activation='sigmoid'):** Final output layer that uses the sigmoid activation function for binary classification.
-

6. COMPILING THE MODEL

```
model.compile(  
    optimizer='adam',                # Adam optimization algorithm  
    loss='binary_crossentropy',      # suitable loss for binary classification  
  
    metrics=['accuracy']             # accuracy as metric  
)
```

Explanation:

- **optimizer='adam':** Optimization algorithm used for training.
 - **loss='binary_crossentropy'** suitable loss metric for binary classification
 - **metrics=['accuracy']:** Accuracy to evaluate the success of the model.
-

7. TRAINING THE MODEL

```
history = model.fit(  
    train_generator,  
    steps_per_epoch=train_generator.samples // train_generator.batch_size,  
    epochs=10,  
    validation_data=validation_generator,  
    validation_steps=validation_generator.samples //  
    validation_generator.batch_size  
)
```

Explanation:

- **steps_per_epoch** Specifies how many steps to take per epoch on the training dataset.
 - **epochs=10** We train the model for 10 epochs.
 - **validation_steps** Determines how many steps to take per epoch on the validation dataset.
-

8. MODEL EVALUATION AND PREDICTION

```
# Evaluate the model's performance on the validation dataset  
val_loss, val_acc = model.evaluate(validation_generator)  
print(f"Validation Accuracy: {val_acc}")
```

```
img_array = image.img_to_array(img) / 255.0 # Normalize the image
img_array = np.expand_dims(img_array, axis=0) # Add batch dimension

prediction = model.predict(img_array)
print("Prediction Probability:", prediction[0][0])
```

Explanation:

- `evaluate()` : Measures the performance of the trained model on the validation dataset.
- `predict()` : We use the model to predict a new example (image).

SUMMARY:

These steps show how to train the ResNet50 model using transfer learning for binary classification:

- We use ResNet50 as a feature extractor.
- Then, we add new layers to our model to perform binary classification.
- We train the model and enhance it with data augmentation.
- Finally, we carry out evaluation and prediction steps.

At every step, we leverage ResNet50's feature extraction capability and train only the final layers to achieve more efficient results with our own dataset.
